TAREEK-Vis: An Interactive Visualization Tool for Large-Scale MATSim Traffic Simulations [Demo]

Jalal Khalil
St. Cloud State University
St. Cloud, MN, USA
jalal.khalil@stcloudstate.edu

Arein Duaibes
College of Saint Benedict & Saint John's University
St. Joseph, MN, USA
aduaibes001@csbsju.edu

Da Yan
Indiana University Bloomington
Bloomington, IN, USA
yanda@iu.edu

Virginia Sisiopiku
University of Alabama at Birmingham
Birmingham, AL, USA
vsisiopi@uab.edu

Abstract

Agent-based traffic simulators such as MATSim produce massive event logs, often several gigabytes of timestamped vehicle movements for a single metropolitan day, that are difficult to inspect, validate, and communicate. Existing visualization tools are either commercial and capped to a few hundred agents in their free tier, or web-based dashboards better suited to aggregate charts than to fluid, frame-accurate animation of every vehicle. We demonstrate *TAREEK-Vis*, a desktop application that replays an entire simulation day at interactive frame rates on commodity hardware. A one-time binary preprocessing step, a compact per-vehicle index, spatial indexing, and GPU-based rendering let the system animate hundreds of thousands of moving agents over the road network while overlaying public-transit lines and stops, background map tiles, and a side-by-side comparison of simulated versus observed link counts. Users can pan, zoom, seek to any moment in time, track individual vehicles, inspect stops and routes, and export figures and videos for reporting. TAREEK-Vis is the visualization companion to TAREEK [3], an automated pipeline our group develops for building MATSim scenarios of arbitrary U.S. metropolitan areas, and it consumes standard MATSim files from any source. We illustrate its use on real MATSim scenarios for four U.S. metropolitan areas, interactively exploring congestion, transit operations, and calibration quality. We will release the source code under an open-source license upon publication.

CCS Concepts

• **Human-centered computing** → **Visualization systems and tools**; • **Information systems** → *Geographic information systems*; • **Applied computing** → *Transportation.*

Keywords

traffic simulation, MATSim, visual analytics, GPU rendering, public transit, spatio-temporal visualization

ACM Reference Format:
Jalal Khalil, Arein Duaibes, Da Yan, and Virginia Sisiopiku. 2026. TAREEK-Vis: An Interactive Visualization Tool for Large-Scale MATSim Traffic Simulations [Demo]. In *Proceedings of The 34th ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL '26).* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Agent-based microsimulation has become the state of the art for transportation planning because it represents every traveler as an autonomous agent with an individual schedule, mode preference, and route choice [2]. A single run of a tool such as MATSim emits an *event log*: a chronologically ordered stream of timestamped records (vehicles entering and leaving links, boarding and alighting at stops, beginning and ending activities) that for a metropolitan area can exceed several gigabytes and hundreds of millions of records for one simulated day.

This richness is also the problem. Raw event logs are effectively unreadable, and the questions practitioners care about are inherently spatio-temporal: *Where and when does congestion form? Do simulated flows match real-world counts? Are transit vehicles running where and when the schedule says they should?* Answering these questions demands a visualization that places agents on the network, plays them through time, and lets the user move fluidly between the city-wide view and a single corridor or vehicle.

The tools available for this task force an uncomfortable trade-off. The dominant commercial viewer, Simunto Via [6], is feature-rich but its free license is capped at 500 agents, restricted to non-commercial use, and expires after six months; unrestricted use requires a paid license. Open-source web-based platforms such as SimWrapper [1] excel at configurable dashboards and aggregate charts, but are oriented toward summary analytics rather than fluid, frame-accurate animation of every individual agent.

We demonstrate *TAREEK-Vis*, a desktop application that removes this trade-off. It is built around a simple principle: convert the verbose XML outputs once into a compact binary representation, then stream and render them on the GPU so that an entire simulation day, with the full agent population of a metropolitan area, plays back smoothly on a laptop and without license restrictions.

SIGSPATIAL '26, Riverside, CA, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

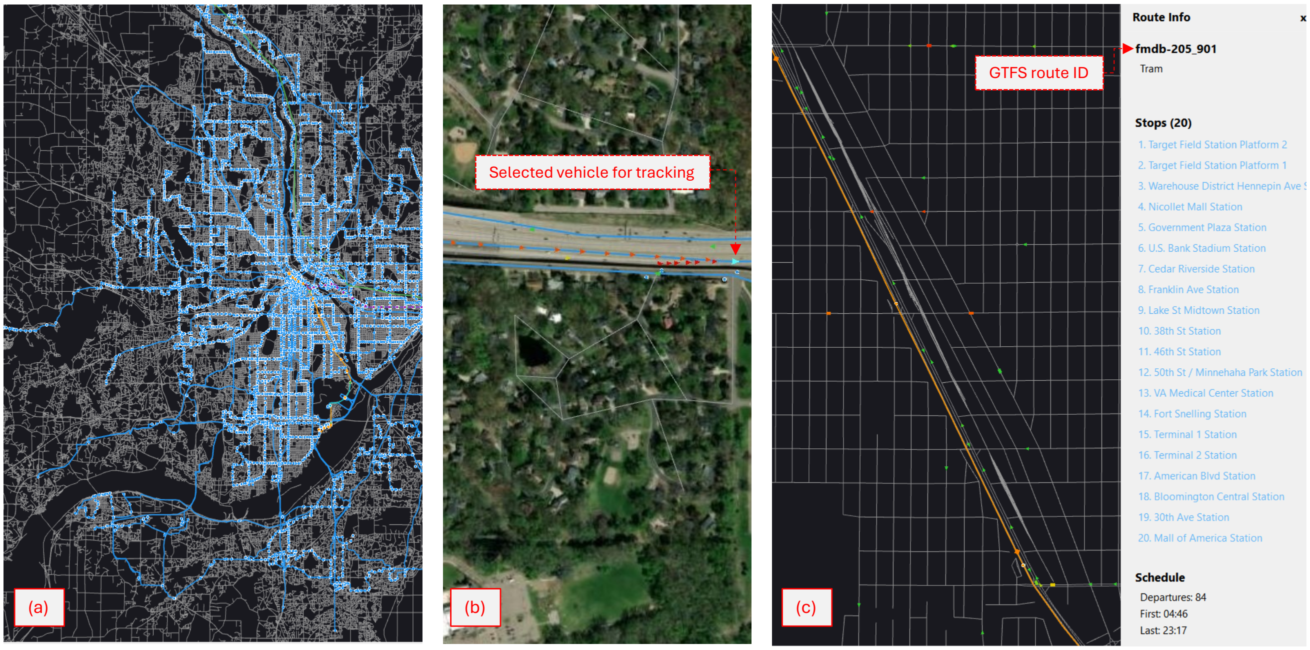

Figure 1: *TAREEK-Vis* on the Twin Cities (MN) scenario. (a) The public-transit network, each tram and rail route a distinct color with dots marking stations. (b) Agents colored by speed, from green (free flow) to red (congested); the one highlighted in light blue is selected and tracked by the camera. (c) A selected tram line and its stops listed in the side information panel.

TAREEK-Vis is the visualization companion to TAREEK [3], an automated framework our group develops that builds and runs MATSim scenarios for arbitrary U.S. metropolitan areas from open data. Its raw outputs are no easier to inspect than any other simulator's; TAREEK-Vis turns them into an interactive, animated map. Because it reads standard MATSim files, it is not limited to TAREEK and works with any MATSim scenario. Its main capabilities are:

- **Smooth replay at scale.** Binary preprocessing, a compact per-vehicle index, spatial indexing, and GPU rendering animate hundreds of thousands of agents over the network at interactive frame rates.
- **Multimodal visualization.** Private vehicles and public-transit vehicles are animated together; transit lines, stops, and routes are rendered as click-to-inspect overlays, with background map tiles for geographic context.
- **Calibration at a glance.** Loading a ground-truth counts file highlights the links with count stations in yellow; clicking one compares its simulated against observed hourly volumes side by side.
- **Reporting.** One-click export of figures (PNG/PDF) and recorded videos of the animation.

2 Related Work

Simunto Via. Via [6] is the most widely used dedicated MATSim visualization tool and offers interactive analyses, animated playback, and plugins for transit, emissions, and traffic counts. Its principal limitation is licensing: the free tier is restricted to 500 agents, permits only non-commercial use, and lapses after six months, while unlimited use requires an educational or commercial license. This places city-scale exploration, the regime in which agent-based models are most useful, behind a paywall. We relied on Via to visualize the MATSim studies in our own prior work [4, 5], and found the educational license, valid for only twelve months, a recurring obstacle: simulations are often run months apart, so a license obtained for one run had frequently expired by the time the next was ready and had to be requested anew. TAREEK-Vis targets the same core task at metropolitan scale and without licensing restrictions.

SimWrapper. SimWrapper [1] is an open-source, web-based platform that turns simulation outputs into configurable dashboards of charts, choropleth maps, and aggregate animations rendered in the browser, with a strength in shareable, reproducible reporting. It is not, however, built for agent-level replay: its event-animation feature is documented as experimental and unreleased [7], loads vehicle data progressively so that seeking to an arbitrary time of day is not possible until loading completes, animates the road and transit networks only separately rather than together, and offers no per-vehicle, stop, route, or count inspection. TAREEK-Vis instead runs as a native desktop application that keeps data local and renders on the GPU, giving it the responsiveness and interaction depth that browser-based aggregate dashboards do not target.

3 System Overview

TAREEK-Vis is a native desktop application written in C++ with a Qt user interface and an OpenGL rendering core. It takes the standard outputs of a MATSim run, a *network* file of road nodes and links, an *events* file containing the timestamped simulation log, and an optional *transit schedule*, and turns them into an interactive, animated map, as shown on the Twin Cities scenario in Figure 1.

(a) New York (NY) **(b) Twin Cities (MN)** **(c) Chicago (IL)** **(d) Birmingham (AL)**

Figure 2: TAREEK-Vis across four U.S. metropolitan areas. Each column is one area; the rows show (top) the rail and tram network, (middle) the bus routes, and (bottom) agent movement at 8 AM during the morning peak. The modeled Birmingham counties have no rail or tram service, so its top panel shows only the road network.

The design follows three ideas that together make city-scale replay smooth on commodity hardware.

Convert once, load instantly. MATSim outputs are stored as compressed XML, which is convenient for archiving but slow to parse repeatedly; a metropolitan event log can take minutes to read on every open. The first time a scenario is opened, TAREEK-Vis streams the XML in a single pass and writes a compact binary cache: the network as fixed-size node and link records, and the raw chronological event stream reorganized into per-vehicle movement segments, so that an agent's trajectory can be reconstructed without scanning the whole log. String identifiers are interned to small integers to shrink the data and speed up lookups. On every subsequent open, the binary cache is read directly into the in-memory layout, turning a minutes-long parse into a near-instant load. The cache lives alongside the scenario and is reused automatically.

Index in time and space. The binary index is organized so that the two operations the user performs most, jumping to a time of day and panning or zooming the map, stay cheap. Along *time*, each vehicle's trajectory is stored as an ordered list of movement segments tagged with their entry and exit times, so the set of agents active at any instant, and each agent's position within its current segment, is recovered by interpolation directly from the index without rereading the event log; seeking to an arbitrary moment is therefore immediate. Along *space*, the road network is stored in a uniform spatial grid, so a pan or zoom redraws only the links and agents inside the current viewport instead of testing the entire network. Because the reorganized index is more than an order of magnitude smaller than the source XML (Section 4), the whole simulated day for a metropolitan area fits entirely in the RAM of a commodity laptop, so playback never has to go back to disk.

Render on the GPU. Rendering is the third bottleneck at city scale, since a single frame may contain hundreds of thousands of moving agents. TAREEK-Vis packs each class of visible element (network links, agents, transit lines, stops, and count overlays) into GPU vertex buffers and draws each class with dedicated shaders. Drawing a whole class in one batched call, rather than one call per object, keeps the number of draw calls the CPU must issue small no matter how many agents are on screen. Agents are interpolated along their current link between consecutive events to produce smooth motion, and are shaded on the GPU either by speed (a green-to-red congestion gradient) or by travel mode (a fixed color per mode, so cars, buses, trams, and rail are visually distinct). Because positioning and coloring happen on the graphics card, the system sustains interactive frame rates while animating hundreds of thousands of simultaneously active agents on a commodity laptop GPU.

On top of this foundation, TAREEK-Vis layers its user-facing features: multimodal animation that distinguishes cars from buses, trams, and rail by shape and color, click-to-inspect transit lines and stops, optional OpenStreetMap, satellite, or topographic background tiles aligned to the network's coordinate reference system, a timeline with variable-speed playback, and the simulated-versus-observed link-count comparison described below. Figures and videos can be exported directly for reporting.

4 Demonstration

We illustrate TAREEK-Vis on real MATSim scenarios for four U.S. metropolitan areas of differing size and transit complexity, namely Birmingham (AL), Twin Cities (MN), Chicago (IL), and New York (NY), shown side by side in Figure 2; it also reads any other MATSim scenario. The walkthrough below traces a typical analysis in six steps. A short companion video at https://youtu.be/Q7_H5vv58Go records the full walkthrough across all four metropolitan areas.

Scale. New York is the largest of the four scenarios and illustrates the design targets. Its simulated day comprises about 1.06 million agents, and the MATSim event log is roughly 27 GB of XML (2.9 GB gzip-compressed). The one-time binary conversion (about five minutes on a laptop) reduces this to a 1.6 GB on-disk index, a ~ 16× shrink over the uncompressed log, which is read back in seconds on every subsequent open and holds the entire simulated day in RAM at once. Playback of this scenario remains fluid and responsive to panning, zooming, and seeking; the companion video shows the New York scenario animating in real time.

(a) Load a scenario. The user selects a city's network and event files (and, if available, its transit schedule). On first load, a progress dialog shows the one-time binary conversion; thereafter the scenario opens almost instantly from cache. The map fits the full network into view, ready to play.

(b) Replay the day and find congestion. Pressing play animates the simulation. Links shift from green toward red as volumes approach capacity, making bottlenecks visually obvious; the morning peak lights up downtown cores and radial arteries in red. Jumping the timeline to any moment and adjusting the playback speed lets the user fast-forward to a peak hour or step slowly through the onset of a jam, and compare how congestion forms across cities of very different scale.

(c) Explore the transit network. With a transit schedule loaded, the user toggles bus, tram, and rail layers, shown as colored polylines with stops as markers. Clicking a stop opens the side information panel with its name, location, and serving lines; clicking a route lists its ordered stops and departure summary and highlights it on the map. Transit vehicles animate along their routes alongside private cars, distinguished by shape and color.

(d) Track an individual agent. Clicking any moving vehicle selects and identifies it, and the camera follows it so that a single agent's progress through the network can be traced. This drill-down from the city-wide view to one vehicle illustrates the per-agent fidelity that distinguishes microsimulation from aggregate flow models.

(e) Compare simulated and observed counts. The user loads a MATSim `counts.xml` file of ground-truth traffic counts. Links that carry a count station are highlighted in yellow, so corridors with observed data stand out at a glance. Clicking one opens a chart placing the simulated hourly volume (red) next to the observed count (green) for all 24 hours of the day (Figure 3). This turns the abstract question "is the simulation calibrated here?" into a direct visual check at any location, revealing where the model over- or under-predicts across the four cities.

(f) Export for reporting. Finally, the current view is exported as a PNG or PDF figure and a short video of the animation recorded, so TAREEK-Vis outputs go directly into papers, reports, and presentations.

All of this runs interactively on a laptop with a commodity GPU, showing that city-scale, agent-level visualization need not be locked behind specialized hardware or paid licenses.

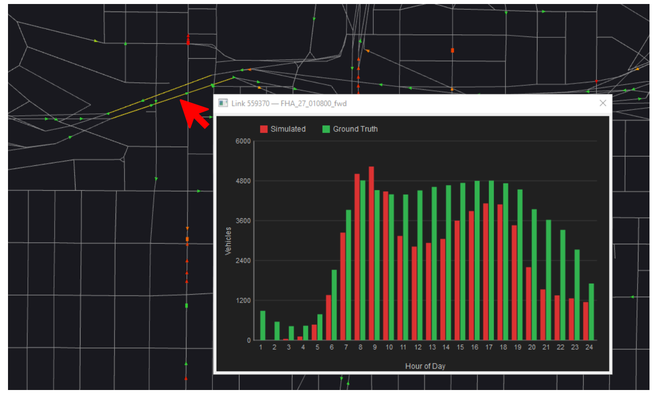

Figure 3: Comparing simulated against observed link counts. Links with a count station are highlighted in yellow (one indicated by the red arrow); clicking one opens its hourly chart of simulated (red) versus ground-truth (green) volumes.

5 Conclusion

TAREEK-Vis makes city-scale MATSim outputs directly explorable: by converting verbose event logs once into a compact binary form and rendering on the GPU, it delivers on commodity hardware the kind of fluid, agent-level exploration that has otherwise required paid licenses or been confined to aggregate dashboards. It serves as the visualization companion to the TAREEK simulation framework while remaining usable with any MATSim scenario, and we will release its source code under an open-source license upon publication. Future work includes congestion and activity-density heatmaps, per-link volume overlays, and visualizing an agent's full daily route across the network.

Acknowledgments

We used generative AI, in particular Anthropic's Claude, as a coding and writing assistant while developing TAREEK-Vis and preparing this manuscript. All design decisions and final text were reviewed and validated by the authors.

References

[1] Billy Charlton and Bhargava Sana. 2023. SimWrapper, an Open Source Web-Based Platform for Interactive Visualization of Microsimulation Outputs and Transport Data. In *Procedia Computer Science*, Vol. 220. Elsevier, 861–866. doi:10.1016/j.procs.2023.03.116

[2] Andreas Horni, Kai Nagel, and Kay W. Axhausen. 2016. *The Multi-Agent Transport Simulation MATSim*. Ubiquity Press, London. doi:10.5334/baw

[3] Jalal Khalil, Arein Duaibes, Da Yan, and Virginia Sisiopiku. 2026. TAREEK: An Automated Traffic Simulation Framework for Any U.S. Metropolitan Area. https://github.com/jalal1/Tareek. Accessed: 2026-06.

[4] Jalal Khalil, Da Yan, Lyuheng Yuan, Saugat Adhikari, Mostafa Jafarzadehfadaki, Virginia Sisiopiku, and Zhe Jiang. 2022. Realistic Urban Traffic Simulation with Ride-Hailing Services: A Revisit to Network Kernel Density Estimation. In *Proceedings of the 30th International Conference on Advances in Geographic Information Systems (SIGSPATIAL '22)*. ACM, 1–12. doi:10.1145/3557915.3560963

[5] Jalal Khalil, Da Yan, Lyuheng Yuan, Mostafa Jafarzadehfadaki, Saugat Adhikari, Jiao Han, Virginia Sisiopiku, and Zhe Jiang. 2024. Urban Traffic Simulation with Shared Mobility Services: An Approach Using Spatiotemporal Network Kernel Density Estimation and MATSim. *ACM Transactions on Spatial Algorithms and Systems* 11 (2024), 1–31. doi:10.1145/3704918

[6] Simunto GmbH. 2025. Via: The Comprehensive Visualization Solution for MATSim. https://www.simunto.com/via/. Accessed: 2026-05.

[7] SimWrapper Team. 2025. MATSim Event Animations. https://docs.simwrapper.app/docs/matsim-events. Under development, not yet released. Accessed: 2026-05.